# SkyHopper: Flame 2D Game Engine Architecture & 60 FPS Loops

## Master Technical Handbook: Beginner to Extreme Senior Architecture

**Project:** SkyHopper 2D Arcade Game
**Engineer:** Akshay Chandran P
**Tech Stack:** Flutter, Dart, Flame Engine, Riverpod, AudioPool, SharedPreferences, AdMob

---

## Section 1: End-to-End Architecture & Working Explanation

### 1.1 The Game Loop: 16.67 Millisecond Frame Budget

Standard apps wait for user input or data events before rebuilding. Games run a continuous **Game Loop** executing 60 to 120 times every second: $$ ext{Frame Budget} = rac{1000 ext{ ms}}{60 ext{ frames}} pprox 16.67 ext{ ms}$$ In Flame, each tick executes two discrete phases: 1. `update(double dt)`: Updates character positions, applies gravity, advances animation clocks, checks collision hitboxes. 2. `render(Canvas canvas)`: Paints the current frame state onto the hardware canvas.

---

### 1.2 Deterministic Movement with Delta Time (`dt`)

If movement were calculated per frame (`position.x += 5`), a player with a 120Hz display would run twice as fast as a player on a 60Hz display!
**The Solution:** Multiply all velocities by `dt` (elapsed time in seconds since the last frame):

```
position.y += velocityY * dt;
```

This guarantees identical physics and movement speeds across all devices regardless of frame rate fluctuations.

---

### 1.3 AudioPool & Zero Allocation in the Game Loop

- Allocating objects (e.g. `Paint()`, `Vector2()`, `List`) inside `update(dt)` triggers the Young Gen Garbage Collector, causing micro-stutter every few seconds.
- Sound effects use `AudioPool` to pre-decode audio files into memory buffers, eliminating latency and memory churn when playing rapid sound effects (jumping, coin collection).

---

*(Handbook continues with spatial hashing collision detection, Riverpod UI overlays, and high-score persistence)*